

MATHEMATICAL FOUNDATIONS

ARTIFICIAL NEURAL NETWORKS

UNIT 3: INTELLIGENT SYSTEMS

$$\frac{\partial}{\partial a} \ln f_{a,\sigma^2}(\xi) = \frac{(\xi - a)}{\sigma^2} f_{a,\sigma^2}(\xi) - \frac{1}{\sigma^2} \ln f_{a,\sigma^2}(\xi)$$

$$\int_{\mathcal{X}} T(x) \frac{\partial}{\partial \theta} f(x, \theta) dx = M \left(T(\xi) \frac{\partial}{\partial \theta} \ln f(\xi, \theta) \right) \int_{\mathcal{X}} f(x, \theta) dx$$

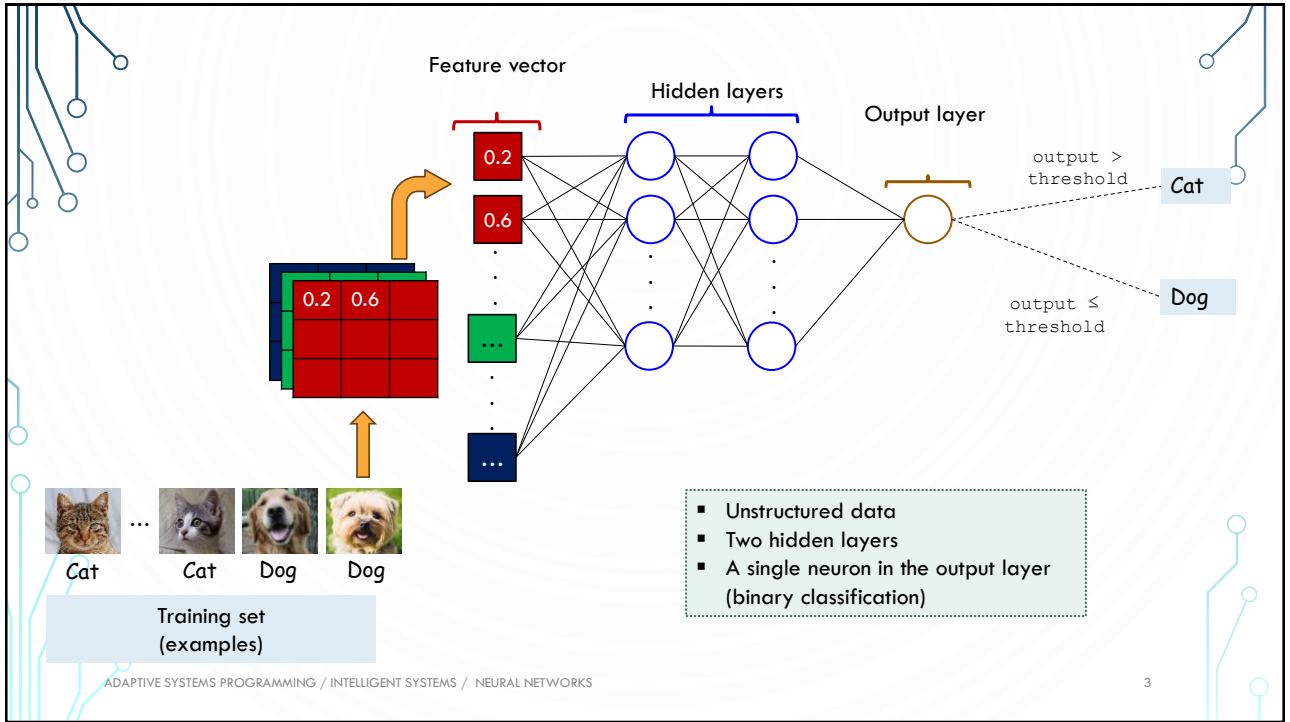
$$\int_{\mathcal{X}} T(x) \left(\frac{\partial}{\partial \theta} \ln f(x, \theta) \right) f(x, \theta) dx = \int_{\mathcal{X}} T(x) \left(\frac{\partial}{\partial \theta} \ln f(x, \theta) \right) f(x, \theta) dx$$

$$\frac{\partial}{\partial \theta} M T(\xi) = \frac{\partial}{\partial \theta} \int_{\mathcal{X}} T(x) f(x, \theta) dx = \int_{\mathcal{X}} T(x) \frac{\partial}{\partial \theta} f(x, \theta) dx$$

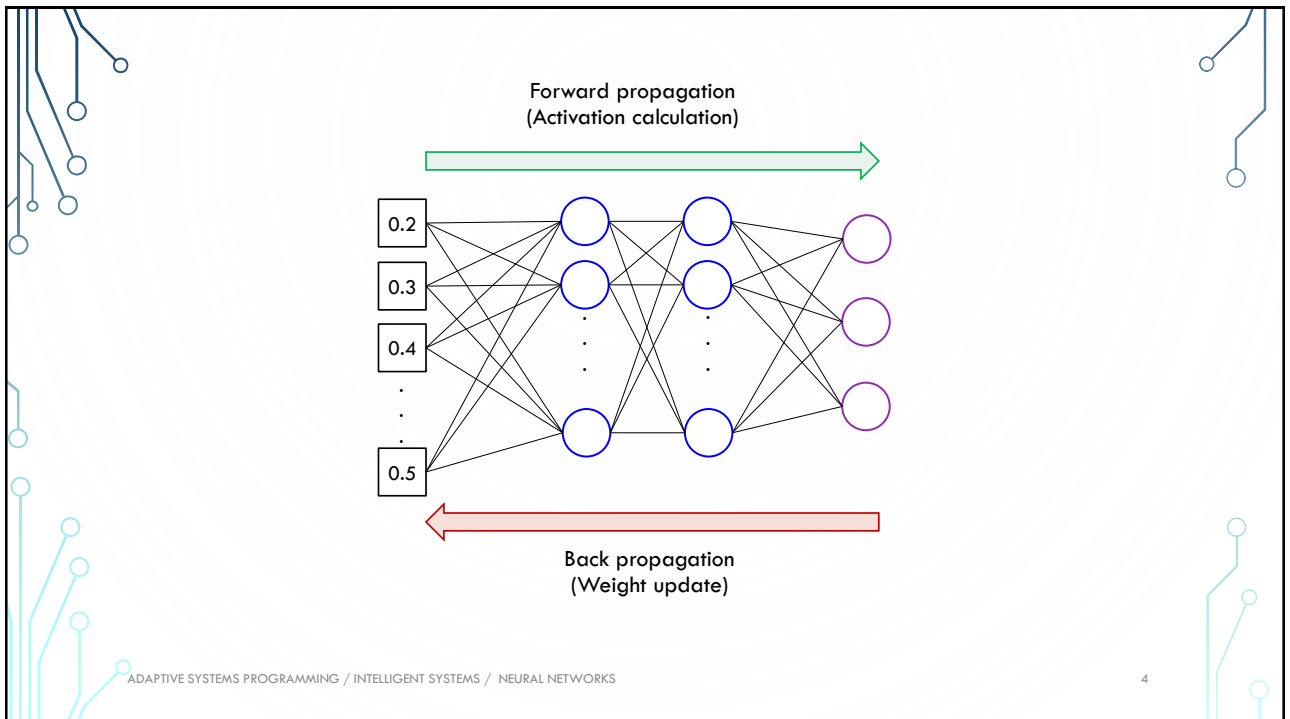
NOTE

- We will stick to the notation used by Andrew Ng
 - Stanford professor
 - Important contributions made to Deep learning
 - Founder of Coursera
 - Creator of the Deep learning specialization



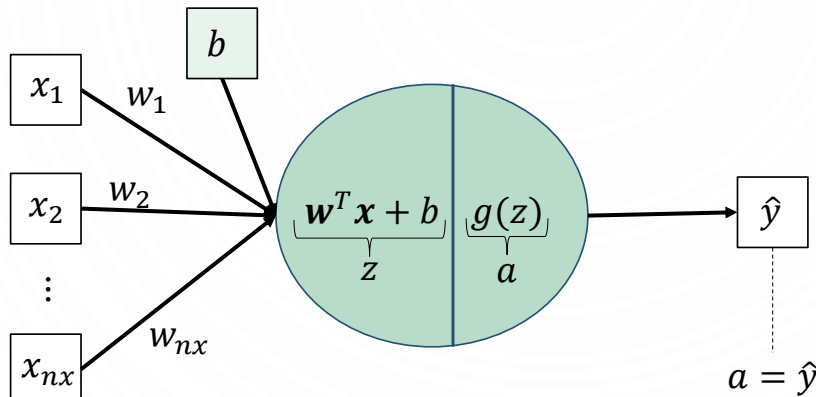


3



4

CASE: 1 EXAMPLE / 1 NEURON

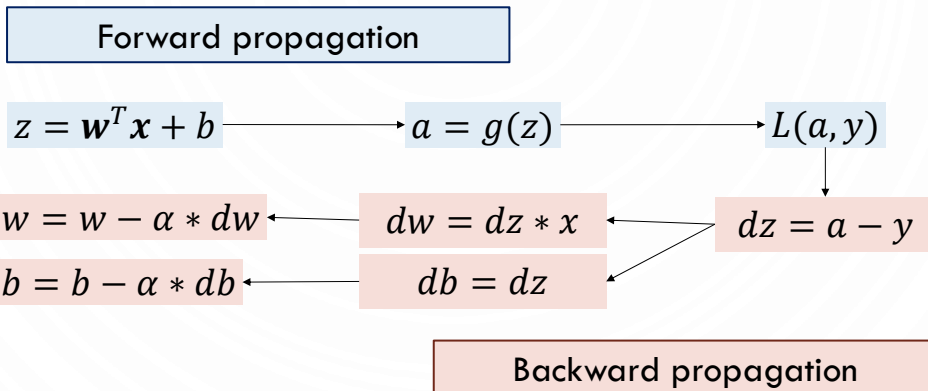


Term	Description
x	Example (feature vector)
w	Weight vector
b	Bias
z	Core result
$g(z)$	Activation result
a	Activation value
$\hat{y}$	Network output

ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

5

TRAINING (FLOW)



ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

6

FORWARD PROPAGATION

CORE / ACTIVATION

ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

7

TRAINING (FLOW)

Forward propagation

$$z = \mathbf{w}^T \mathbf{x} + b$$

Core

$$a = g(z)$$

Activation

$$L(a, y)$$

Error

ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

8

PYTHON TOOLS

VECTORIZATION

- Faster than loops.

BROADCASTING

- Permits to use scalars and vectors for matrix operations.

INPUTS (TRAINING EXAMPLES)

 $\mathbf{x}$

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{nx} \end{bmatrix}$$

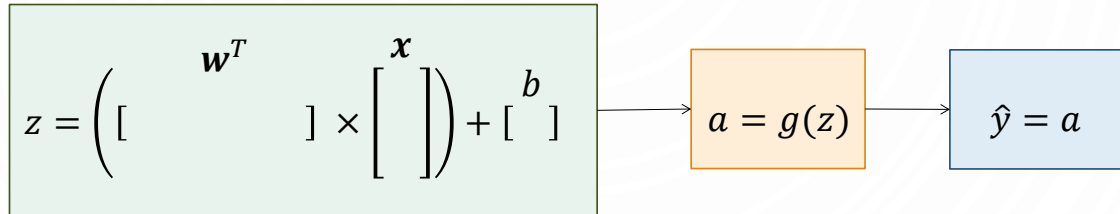
Feature vector
(example)

 $\mathbf{X}$

$$\begin{bmatrix} x_1^{(1)} & \cdots & x_1^{(m)} \\ \vdots & \ddots & \vdots \\ x_{nx}^{(1)} & \cdots & x_{nx}^{(m)} \end{bmatrix}$$

Matrix of examples

- Example i is denoted as $x^{(i)}$
- Feature j of example i is denoted as $x_j^{(i)}$
- The training set is denoted as X
- There are nx features
- There are m examples



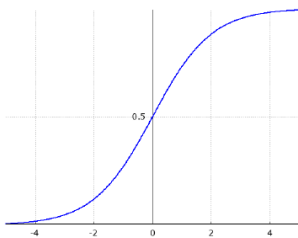
ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

11

ACTIVATION FUNCTIONS

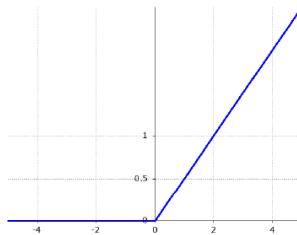
Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



ReLU

$$R(z) = \max(0, z)$$



- There are several types of activation functions ($g(z)$). We will use sigmoid and ReLU.

Other

- Hyperbolic tangent
- Leaky ReLU
- Softmax

ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

12

EXAMPLE (CALCULATING THE OUTPUT)

Core	Activation	Output
$z = \left([1 \ 2 \ 3] \times \begin{bmatrix} 1 \\ -2 \\ -1 \end{bmatrix} \right) + [1.5]$ $z = (-6) + 1.5$ $z = -4.5$	$a = g(z)$ $a = \sigma(z)$ $a = \frac{1}{1 + e^{-z}}$ $a = \frac{1}{1 + e^{4.5}}$ $a = 0.011$	$\hat{y} = 0.011$

ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

13

CALCULATING ERROR (LOSS)

Recalling that $\hat{y} = a$

This kind of loss function is called *cross entropy*

$$L(\hat{y}, y) = -(y \ln \hat{y} + (1 - y) \ln(1 - \hat{y}))$$

ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

14

EXAMPLE (CALCULATING THE ERROR)

$$\hat{y} = 0.011$$

$$y = 1$$

$$L(\hat{y}, y) = -(y \ln \hat{y} + (1 - y) \ln(1 - \hat{y}))$$

$$L(0.011, 1) = -((1) \ln(0.011) + (1 - 1) \ln(1 - 0.011))$$

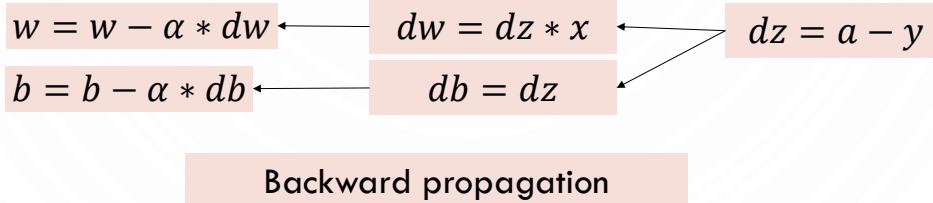
$$= -((1) \ln(0.011))$$

$$= -4.51$$

BACK-PROPAGATION

GRADIENTS / WEIGHT UPDATE

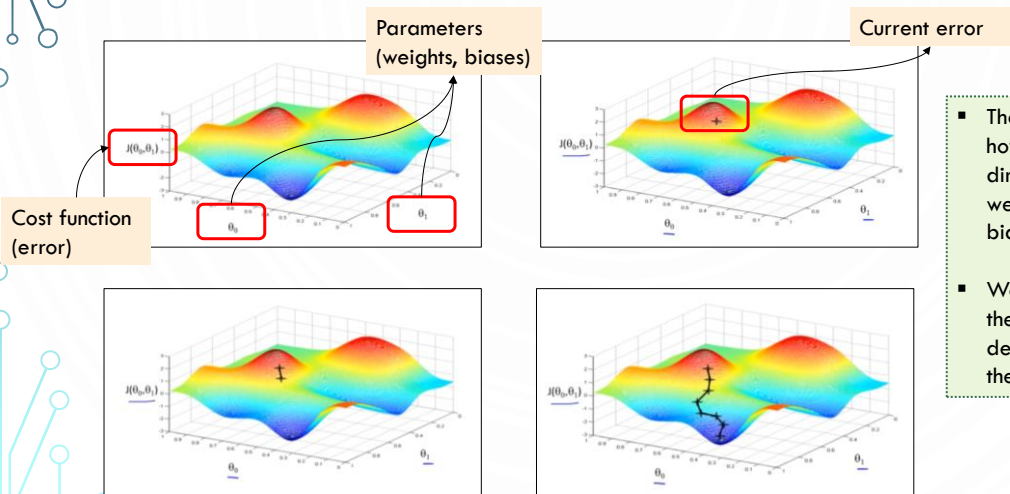
TRAINING (FLOW)



ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

17

GRADIENT DESCENT



- The gradient tells us how much and in which direction to move the weights (as well as the biases).
- We attempt to minimize the error (get into the deepest valley) by using the gradient.

ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

18

GRADIENT DESCENT

Gradient	Simplified notation	Formula
$\frac{\partial L}{\partial z}$	dz	$dz = a - y$
$\frac{\partial L}{\partial w}$	dw	$dw = dz * x$
$\frac{\partial L}{\partial b}$	db	$db = dz$

EXAMPLE (GRADIENT CALCULATION)

$$\hat{y} = a = 0.011$$

$$y = 1$$

$$dw = \begin{bmatrix} -0.989 \\ -0.989 \\ -0.989 \end{bmatrix} * \begin{bmatrix} 1 \\ -2 \\ -1 \end{bmatrix} = \begin{bmatrix} -0.989 \\ 1.978 \\ 0.989 \end{bmatrix}$$

$$dz = 0.011 - 1$$

$$dz = -0.989$$

$$db = -0.989$$

UPDATING WEIGHTS

$$w = w - \alpha * dw$$

$$b = b - \alpha * db$$

- α : learning rate
- Learning “velocity”

EXAMPLE (WEIGHT UPDATE)

$$w = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} - \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix} * \begin{bmatrix} -0.989 \\ 1.978 \\ 0.989 \end{bmatrix}$$

$$w = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} - \begin{bmatrix} -0.0989 \\ 0.1978 \\ 0.0989 \end{bmatrix}$$

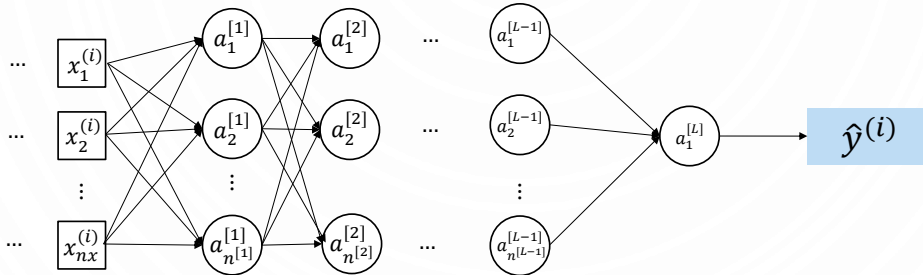
$$w = \begin{bmatrix} 1.0989 \\ 1.802 \\ 2.9011 \end{bmatrix}$$

$$b = 1.5 - (0.1) * (-0.989)$$

$$b = 1.5 - (-0.0989)$$

$$b = 1.5989$$

GENERAL CASE



ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

23

SUMMARY

FORMULAS

ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

24

FORMULAS (SINGLE NEURON)

Equation	Formula	Process	Description
1	$z = \mathbf{w}^T \mathbf{x} + b$	FP	Core result
2	$a = g(z)$	FP	Activation function
3	$g(z) = \sigma(z)$	FP	Means that the sigmoidal function will be used as activation function.
4	$g(z) = R(z)$	FP	Means that the ReLU function will be used as activation function.
5	$\sigma(z) = \frac{1}{1 + e^{-z}}$	FP	Sigmoidal activation function
6	$R(z) = \max(0, z)$	FP	ReLU activation function

ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

25

FORMULAS (SINGLE NEURON)

Equation	Formula	Process	Description
7	$dz = a - y$	BP	Change in error w.r.t. core
8	$dw = dz * x$	BP	Change in error w.r.t. weights
9	$db = dz$	BP	Change in error w.r.t. bias
10	$w = w - \alpha * dw$	BP	Weight update
11	$b = b - \alpha * db$	BP	Bias update

Eq.		
12	$L(\hat{y}, y) = -(y \ln \hat{y} + (1 - y) \ln(1 - \hat{y}))$	Error (loss) calculation

ADAPTIVE SYSTEMS PROGRAMMING / INTELLIGENT SYSTEMS / NEURAL NETWORKS

26